Improving Bike Rental Demand Prediction Through Data Augmentation: Vanilla Training, Mixup and RegMix Compared

Amit Jadhav, Atharva Jamathkar, Yashvardhan Agwan
Department of Computer Engineering (Regional Language), Pimpri Chinchwad College of Engineering (PCCOE), Pune, India

***Abstract*—Short-term forecasts of bike rental demand help operators allocate fleets and plan their work, but the models behind them are often fitted to only a small number of labelled observations. Data augmentation can regularise such models. Mixup, however, was designed for classification: it interpolates labels linearly, and when two mixed examples lie far apart in a continuous label space the resulting target can be misleading. RegMix (Hwang and Whang) tackles this by learning, separately for each training example, how many nearest neighbours it may be mixed with. This study compares Vanilla training, Mixup, a global nearest-neighbour mixing baseline and a reduced-budget re-implementation of RegMix (40 search iterations) on the hourly UCI Bike Sharing data. All methods share one multilayer perceptron and are evaluated under a reproduction-style random-subset protocol and a chronological protocol, at four training-set sizes and over five random seeds, using RMSE, MAE and R^2. Augmentation helped little. With a random training set of 30 examples, RegMix lowered the mean RMSE from 153.6 to 144.4 bikes per hour and won in all five seeds, but this was not significant after Holm correction. At 300 examples Vanilla training was best (RMSE 105.0) and RegMix worst (115.7). Under chronological evaluation no augmentation method beat Vanilla at any size, including RegMix, which was run at all four sizes. RegMix took about 8 to 14 times longer to train than Vanilla. The findings apply to this dataset and to our reduced search budget.**

***Index Terms*—bike-sharing demand prediction; data augmentation for regression; Mixup; RegMix; multilayer perceptron; chronological evaluation.**

I. Introduction

A bike-sharing system only works when bikes are present where and when riders want them, so operators rely on demand forecasts to allocate and rebalance their fleets. A systematic review of 35 papers from 2015 to 2019 grouped machine-learning research on bike sharing into classification and prediction tasks [1]. Earlier studies examined the factors that drive bike-sharing demand [2] and used past usage patterns to predict the availability of bikes [3]. Demand models are, however, often fitted to a modest number of labelled observations, and the influence of weather and calendar variables on demand is nonlinear. In this situation regularisation is important, and data augmentation offers a way to generalise better without collecting additional data. Augmentation is well established for images [4] and has been surveyed for time series [5], [6]; adding noise to the inputs is a classical form of regularisation [7].

Mixup builds synthetic examples as linear combinations of two training inputs and of their labels, with the mixing weight sampled from a Beta distribution [8]. It was proposed for classification, and theoretical work has examined why it regularises [9], [10]. Variants such as Manifold Mixup [11] and CutMix [12] also target classification, and Mixup can be seen as a form of vicinal risk minimisation [13]. With a continuous label space, mixing two distant examples can produce a label that does not match the mixed input, a problem related to manifold intrusion [14]. RegMix [15] addresses this by restricting how far apart mixed examples may be. A controller trained by reinforcement learning selects, for each training example, the number of nearest neighbours it can be mixed with, and uses the validation loss as its reward. This follows the idea of searching for augmentation policies with a learned controller, as in AutoAugment [16], Fast AutoAugment [17] and neural architecture search [18]. C-Mixup [19] shares the concern but takes another route, choosing mixing partners by similarity of their labels.

RegMix is a prominent method for this problem in tabular regression, yet the evidence for it on bike demand is limited. In [15], Bike is one of four datasets, is evaluated on a random subset of 300 training and 200 validation examples, and has no chronological evaluation. The authors also note that RegMix needs hours to converge, so its cost matters for practical use. Three questions therefore remain open: how the method behaves when the test period lies in the future, how its benefit varies with training-set size, and whether a simpler global nearest-neighbour restriction achieves most of the improvement.

This paper examines these questions through a controlled comparison on the UCI Bike Sharing dataset [20], whose data were first described in [21]. We ask: (RQ1) does augmentation improve prediction compared with training without it; (RQ2) does selective nearest-neighbour mixing outperform unrestricted Mixup; (RQ3) how do the answers change with training-set size; (RQ4) are the differences consistent across random seeds and under chronological rather than random evaluation; and (RQ5) what computational cost does the learned policy add. The contribution is an evaluation under one consistent protocol rather than a new algorithm. The findings concern this dataset, a single system observed over two years, and are not claimed to carry over to other cities.

II. Methods

Fig. 1 summarises the study design. The same preprocessing, network and splits are used for every method, and only the augmentation step differs.

A. Data

Only the Bike Sharing dataset from the UCI Machine Learning Repository [20] is used. It holds hourly and daily rental counts for the Capital Bikeshare system over 2011 and 2012, together with weather and seasonal information. We use the hourly file, hour.csv. Its inputs are the calendar variables (season, year, month, hour, holiday, weekday, working day), a weather-situation category, and four normalised weather

quantities: temperature, apparent temperature, humidity and wind speed. According to the dataset documentation, temperature is scaled as $(t - t_{min})/(t_{max} - t_{min})$ with $t_{min} = -8$ °C and $t_{max} = 39$ °C, humidity is divided by 100, and wind speed by 67 [20]. The prediction target is cnt, the total rental count, which is the sum of the casual and registered counts. Because both components would leak the target, neither is used as an input, and the record index and date string are also removed. The documentation states that no values are missing; we confirm this, together with a duplicate check and an outlier screen, on the downloaded file (Table I). Outliers are kept because hours of very high demand are genuine observations.

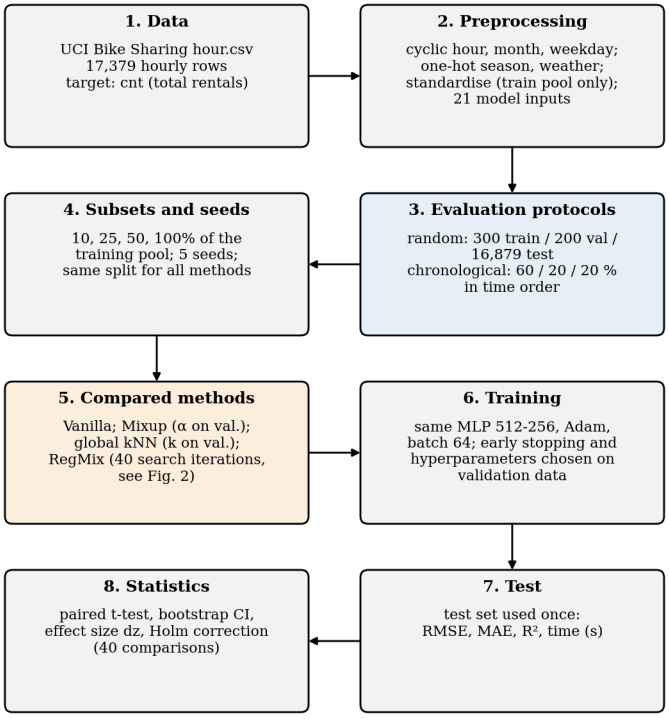


Fig. 1. Overview of the study design: data, preprocessing, the two evaluation protocols, training subsets and seeds, the four compared methods, shared training and validation-based selection, a single test evaluation and paired statistics.

TABLE I
DATASET SUMMARY (MEASURED ON THE DOWNLOADED FILE)

Item	Value
Rows in hour.csv	17,379
Model inputs after encoding	21
Missing values	0
Duplicate rows	0
Target mean / std / max	189.5 / 181.4 / 977
Target values outside IQR fences	505 (2.9%)

B. Preprocessing

Hour, month and weekday are represented cyclically with sine and cosine terms. Season and weather situation are one-hot encoded, while year, holiday and working day remain binary. The four weather quantities are used as provided. All inputs are then standardised, with the scaler fitted on the training pool only. The target is standardised using statistics from the labelled training subset, and every reported error is converted back to bike counts. Preprocessing is identical for all methods.

C. Evaluation Protocols

We use two protocols, which answer different questions. The reproduction-style protocol follows the Bike setting of [15]: 300 training and 200 validation examples are drawn at random, and all remaining 16,879 rows form the test set. The composition of the original test set is not specified in [15], so this is an assumption. The chronological protocol orders records by time and assigns the first 60% (10,427 rows) to training, the next 20% to validation and the final 20% to testing, so the test period comes after the training period. Evaluating on a later period avoids testing on observations that precede the training data; the use of cross-validation for time-series predictors is discussed in [22]. Under each protocol, subsets of 10%, 25%, 50% and 100% of the training pool are drawn from the training pool alone (30, 75, 150 and 300 examples for the reproduction-style protocol; 1,043, 2,607, 5,214 and 10,427 for the chronological protocol), and validation and test rows are never used as mixing inputs. The validation data serve to choose the learning rate and the Mixup and global-neighbour hyperparameters, to trigger early stopping, and to learn the RegMix policy. The test set is used once, for the final evaluation. Each configuration is repeated over five random seeds (0 to 4), and all methods share the same split within a seed.

D. Regression Model

All methods use one multilayer perceptron with hidden layers of 512 and 256 units, ReLU activations, a mean-squared-error loss, the Adam optimiser [23] and mini-batches of 64. This follows the architecture used for Bike in [15]. Tree ensembles [24], [25], [26] and recurrent networks [27] are common alternatives for tabular and sequential data, but we keep a single network so that only the augmentation differs. The learning rate is chosen from $\{1\times10^{-4}, 3\times10^{-4}, 1\times10^{-3}\}$ on validation error using Vanilla training only, on the full training pool of the first seed, so that the selection does not favour augmentation. It is selected once per protocol and then held fixed: 1×10^{-3} for the reproduction-style protocol and 3×10^{-4} for the chronological protocol. Validation error is evaluated every 100 steps, training stops after 30 evaluations without improvement or after 20,000 steps, and the weights with the best validation error are restored.

E. Compared Methods

Vanilla. Training on the original examples only.

Mixup. For a training example (x_i, y_i) and a partner (x_j, y_j) drawn uniformly from the remaining training examples, the synthetic example is $\tilde{x} = \lambda x_i + (1 - \lambda)x_j$ with target $\tilde{y} = \lambda y_i + (1 - \lambda)y_j$. The weight λ is sampled from Beta(α, α) and is shared by input and target [8]. The concentration α is selected from {0.2, 1, 10, 100} on validation data.

Global kNN mixing. The partner is drawn uniformly from the k nearest training neighbours (Euclidean distance on standardised features). A single k is shared by all examples and selected on validation data from {1, 4, 16, 64, 256}, capped at $n - 1$ for small training sets. This serves as a simplified baseline derived from [15].

RegMix. Following [15], a controller assigns every training example a neighbour count from {0, 1, 4, 16, 64, 256}, where 0 means no mixing (capped at $n - 1$ for small training sets), and mixing weights are drawn from Beta(100, 100). The

controller is a multilayer perceptron with four hidden layers of 100 units. Its input is a vector of ones and it outputs one softmax per example. In each iteration twenty policies are sampled and a regression model is trained under each. The reward for a policy is the inverse of its validation loss minus an exponential moving average baseline (weight 0.95). The controller is updated with Proximal Policy Optimization [28] using four epochs per iteration, a clipping value of 0.2, a learning rate of 0.002 and an entropy bonus of 0.01. Fig. 2 summarises the search loop.

The search budget used in [15], which takes hours of computation, is not available to us. We therefore call our method a reduced-budget re-implementation rather than a replication. The main differences are as follows. The policy search runs for only 40 iterations with twenty policies sampled per iteration, and each candidate model is trained for a fixed 1,500 steps without early stopping. The paper [15] does not state a fixed iteration count, but its reward curves (Figs. 7b and 8b of [15]) run for up to 300 episodes. Mixed examples are generated on the fly at every step from the neighbour set that an example's policy selects, instead of materialising all pairs. Details of PPO that [15] does not specify (the per-example probability ratio and the advantage scaling) are our own choices. The policy used for the final model is the controller's most probable option for each example. RegMix was run at all four training sizes under both protocols. The two largest chronological sizes (5,214 and 10,427 rows) were run in a second session with the same settings, which also repeated the other three methods for those sizes and reproduced their earlier RMSE values.

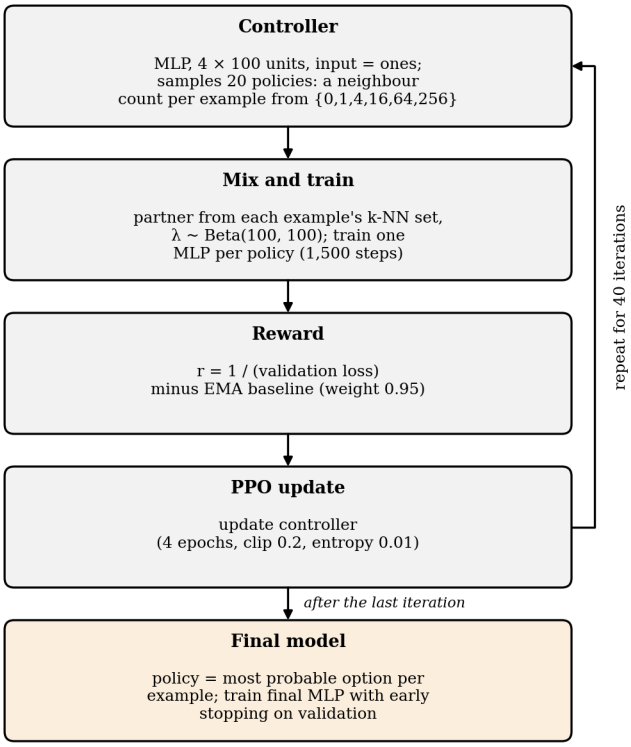


Fig. 2. RegMix policy search as implemented here. Each iteration samples 20 policies, trains one model per policy, converts the validation losses to rewards and updates the controller with PPO.

F. Metrics and Statistical Analysis

Let y_i be the test targets, $\hat{y}_i$ the predictions, $\bar{y}$ their mean and n the number of test observations. Errors are reported in bikes per hour:

$$\mathrm{RMSE} = \sqrt{\left(\tfrac{1}{n}\right) \Sigma_i \, (y_i - \hat{y}_i)^2} \quad (1)$$

$$\mathrm{MAE} = \left(\tfrac{1}{n}\right) \Sigma_i \, |y_i - \hat{y}_i| \quad (2)$$

$$R^2 = 1 - \Sigma_i \, (y_i - \hat{y}_i)^2 \, / \, \Sigma_i \, (y_i - \bar{y})^2 \quad (3)$$

Results are given as the mean and standard deviation over the five seeds. Methods are compared using paired tests across seeds on identical splits: a paired t-test, a Wilcoxon signed-rank test [29], a bootstrap 95% confidence interval [30] for the mean RMSE difference, and the paired effect size d_z, following common practice for comparing learning algorithms [31]. Holm correction [32] is applied to the t-test p-values across all 40 paired comparisons. A two-sided Wilcoxon test cannot reach $p < 0.05$ with fewer than six pairs (with five pairs its smallest possible p-value is 0.0625), so the Wilcoxon p-values are descriptive only. Statistical significance is kept separate from practical importance. Training time is measured as wall-clock seconds on a single Tesla T4 GPU and, for RegMix, includes the policy search and the neighbour search. The code is written in Python with PyTorch 2.11 [33], using scikit-learn [34] for the metrics and SciPy [35] for the statistical tests, and run on Google Colab, and seeds, configuration and software versions are stored with the results.

III. Results and Discussion

A. Data Characteristics

Fig. 3 shows a clear daily pattern. Demand is lowest between about 02:00 and 05:00, rises to a morning peak at 08:00, dips around 10:00, and reaches its highest point at 17:00 to 18:00, which suggests commuting. The evening peak averages roughly 570 rentals per hour in summer and about 265 in winter, so season mainly scales the same daily shape. Overall the target has a mean of 189.5 and a standard deviation of 181.4 bikes per hour, with a maximum of 977 (Table I).

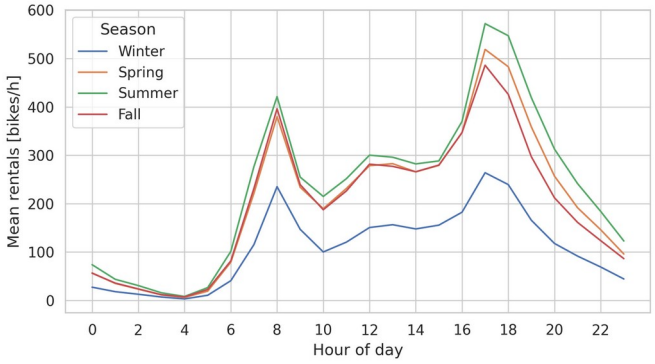


Fig. 3. Mean hourly rentals by hour of day and season (bikes per hour).

Fig. 4 shows that rentals correlate most strongly with temperature and apparent temperature ($r = 0.40$ for both), then with the hour index ($r = 0.39$), humidity ($r = -0.32$) and year ($r = 0.25$). Holiday, weekday and working day have almost no linear correlation with rentals ($|r| \leq 0.03$). Temperature and apparent temperature are almost identical ($r = 0.99$) and season and month are strongly related ($r = 0.83$), so these inputs carry redundant information. The linear correlation with the hour index understates its importance because the daily pattern is not monotonic. Temperature is also confounded with the time of day and season (Fig. 3), so the correlations should not be read as the effect of weather alone.

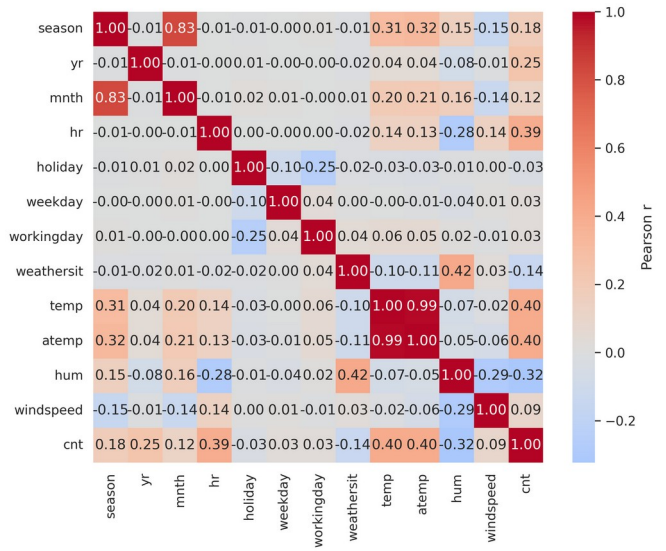


Fig. 4. Pearson correlation between inputs and rentals. Correlation does not imply causation.

B. Results Reported in the Original RegMix Study

For context, Table II lists the Bike results published by the RegMix authors [15]. They were obtained in the authors' own setting (a random subset of 300 training and 200 validation examples, five repetitions) and are not results of the present study. Their RMSE values, roughly 369 to 393, exceed the standard deviation of our target (181.4 bikes per hour, Table I), which is roughly the error of a predictor that always outputs the mean. The setting in [15] therefore differs from ours in some way that [15] does not specify (for example the target scale, the file or the preprocessing), and we do not compare our RMSE values with Table II numerically.

TABLE II
BIKE RESULTS PUBLISHED BY HWANG AND WHANG [15] (NOT OBTAINED IN THIS STUDY)

Method (as reported in [15])	RMSE (mean ± sd)
Vanilla	393.45 ± 0.90
Mixup	393.36 ± 1.84
Global kNN (k = 68)	388.81 ± 1.41
RegMix	368.86 ± 3.07

C. Main Comparison

Table III gives test performance at the full training size of each protocol. Under the reproduction-style protocol (300 training examples), Vanilla training had the lowest RMSE (104.97 ± 3.81) and global kNN mixing was close behind (106.62 ± 4.95). Mixup (113.24 ± 4.00) and RegMix (115.67 ± 4.34) were worse. Relative to Vanilla, RegMix was 10.7 bikes per hour worse (95% CI 9.9 to 11.8; Holm-adjusted p = 0.002) and Mixup 8.3 worse (Holm-adjusted p = 0.102), while the difference between Vanilla and global kNN was within noise (Table V). MAE and R^2 gave the same ordering. Under the chronological protocol (10,427 training rows), Vanilla again had the lowest RMSE (88.60 ± 1.92), followed by global kNN (90.74 ± 2.00), Mixup (91.62 ± 2.68) and RegMix (95.05 ± 6.00), which also varied most between seeds. The gaps to Vanilla (2.1, 3.0 and 6.5 bikes per hour) were not significant after correction. RMSE values are lower under the chronological protocol, but the two protocols use different test sets, so their errors are not directly comparable.

TABLE III
TEST PERFORMANCE AT THE FULL TRAINING SIZE UNDER EACH PROTOCOL (MEAN ± SD OVER FIVE SEEDS; RMSE AND MAE IN BIKES PER HOUR; BEST PER PROTOCOL IN BOLD)

Method	RMSE	MAE	R^2	Time (s)
Reproduction-style protocol (300 training examples)				
Vanilla	**104.97 ± 3.81**	**66.90 ± 2.67**	**0.665 ± 0.024**	20.67 ± 8.08
Mixup	113.24 ± 4.00	75.73 ± 3.10	0.610 ± 0.027	7.72 ± 0.36
Global kNN	106.62 ± 4.95	68.72 ± 4.36	0.654 ± 0.031	32.69 ± 15.09
RegMix	115.67 ± 4.34	76.79 ± 2.83	0.593 ± 0.030	163.76 ± 2.36
Chronological protocol (10,427 training rows)				
Vanilla	**88.60 ± 1.92**	**59.79 ± 1.36**	**0.838 ± 0.007**	12.01 ± 3.03
Mixup	91.62 ± 2.68	62.24 ± 1.94	0.827 ± 0.010	19.31 ± 2.57
Global kNN	90.74 ± 2.00	62.15 ± 4.23	0.831 ± 0.008	25.45 ± 7.19
RegMix	95.05 ± 6.00	62.33 ± 3.48	0.814 ± 0.024	172.70 ± 2.17

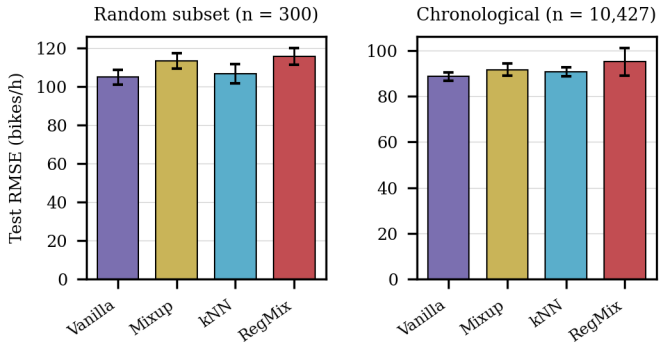


Fig. 5. Test RMSE by method under each protocol at the full training size (error bars: standard deviation over five seeds).

D. Effect of Training-Set Size

Table IV and Fig. 6 show how the ranking changes with training-set size. Under the reproduction-style protocol, RegMix had the lowest mean RMSE at 30 examples (144.42 versus 153.58 for Vanilla) and at 75 examples (135.52 versus 138.80). At 30 examples it beat Vanilla in all five seeds (mean gain 9.2 bikes per hour, 95% CI 4.6 to 14.0; unadjusted paired t-test p = 0.028; Wilcoxon p = 0.0625, the smallest value possible with five pairs). However, the Holm-adjusted p-value across all 40 comparisons was 0.80, so we treat this as suggestive rather than established. At 75 examples RegMix beat Vanilla in four of five seeds and the gain was smaller (3.3; CI −0.1 to 6.1). At 150 examples the methods were within about 3.5 bikes per hour of each other, and at 300 examples RegMix was the worst method. Mixup and global kNN improved on Vanilla only at 30 examples (by about 2.6 and 2.1 bikes per hour) and beat it in only two of five seeds.

Under the chronological protocol Vanilla had the lowest mean RMSE at every size. Mixup was worse than Vanilla by 3.5, 4.7, 6.2 and 3.0 bikes per hour at the four sizes, and global kNN by 5.8, 12.8, 11.9 and 2.1. RegMix was worse than Vanilla by 6.1, 11.9, 14.0 and 6.5 bikes per hour at the four sizes (Holm-adjusted p = 0.62, 0.52, 0.75 and 1.00). Error fell with more training data for all methods under both protocols.

TABLE IV

TEST RMSE BY TRAINING-SET SIZE UNDER EACH PROTOCOL (BIKES PER HOUR; MEAN ± SD OVER FIVE SEEDS; BEST IN BOLD)

Size	Vanilla	Mixup	Global kNN	RegMix
Reproduction-style protocol				
10% (30)	153.58 ± 6.67	150.96 ± 9.27	151.51 ± 8.08	**144.42 ± 9.17**
25% (75)	138.80 ± 4.58	140.65 ± 8.77	139.43 ± 4.63	**135.52 ± 3.20**
50% (150)	**128.20 ± 10.51**	131.69 ± 7.20	129.55 ± 7.58	129.90 ± 4.81
100% (300)	**104.97 ± 3.81**	113.24 ± 4.00	106.62 ± 4.95	115.67 ± 4.34
Chronological protocol				
10% (1,043)	**114.70 ± 8.77**	118.24 ± 3.21	120.49 ± 9.80	120.75 ± 5.52
25% (2,607)	**96.64 ± 5.59**	101.34 ± 1.90	109.46 ± 4.83	108.51 ± 10.95
50% (5,214)	**89.32 ± 4.54**	95.56 ± 3.41	101.23 ± 8.82	103.35 ± 10.80
100% (10,427)	**88.60 ± 1.92**	91.62 ± 2.68	90.74 ± 2.00	95.05 ± 6.00

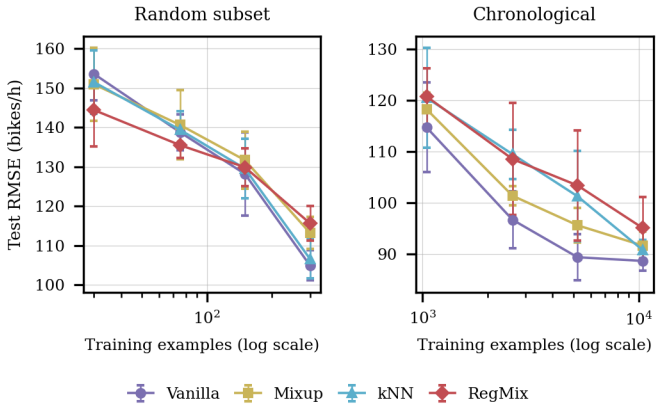


Fig. 6. Test RMSE versus training-set size (log scale) for each protocol (error bars: standard deviation over five seeds).

E. Consistency Across Seeds and Statistical Analysis

Fig. 7 shows the individual seeds at the full training size and Table V lists selected paired comparisons. Seed-to-seed standard deviations of RMSE at the full size were 3.8 to 5.0 bikes per hour under the reproduction-style protocol and 1.9 to 6.0 under the chronological protocol (the largest, 6.0, for RegMix), so differences smaller than these are not reliable. With five seeds only the paired t-test and the bootstrap intervals are informative, and Holm correction over 40 comparisons is conservative. Only one comparison remained significant at the 0.05 level after correction: RegMix being worse than Vanilla at 300 examples. Several other comparisons have confidence intervals that exclude zero but did not survive correction, including RegMix being better than Vanilla at 30 examples Mixup and global kNN being worse than Vanilla at 10,427 rows, and RegMix being worse than Vanilla at all four chronological sizes. Consistency across seeds supports the same picture: Mixup, global kNN and RegMix beat Vanilla in at most one of five seeds at every chronological size, and RegMix beat Vanilla in five of five seeds at 30 examples but in none at 300.

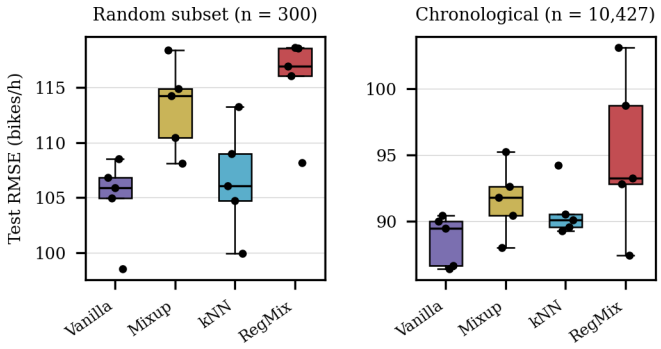


Fig. 7. Variability of test RMSE across five random seeds at the full training size (points: individual seeds).

TABLE V

SELECTED PAIRED COMPARISONS (FIVE SEEDS). GAIN IS THE RMSE REDUCTION OF THE FIRST-NAMED METHOD RELATIVE TO THE SECOND, SO NEGATIVE VALUES MEAN THE FIRST METHOD WAS WORSE. HOLM: ADJUSTED OVER ALL 40 COMPARISONS

Comparison	Gain	95% CI	d_z	Holm p
Mixup vs Vanilla (random, 300)	−8.3	[−10.3, −5.9]	−3.0	0.102
kNN vs Vanilla (random, 300)	−1.7	[−3.8, 0.5]	−0.6	1.000
RegMix vs Vanilla (random, 300)	−10.7	[−11.8, −9.9]	−8.6	0.002
RegMix vs Mixup (random, 300)	−2.4	[−5.5, 0.2]	−0.7	1.000
RegMix vs kNN (random, 300)	−9.0	[−11.9, −6.6]	−2.7	0.144
RegMix vs Vanilla (random, 30)	9.2	[4.6, 14.0]	1.5	0.799
RegMix vs Vanilla (random, 75)	3.3	[−0.1, 6.1]	0.8	1.000
Mixup vs Vanilla (chrono, 10,427)	−3.0	[−5.5, −0.04]	−0.9	1.000
kNN vs Vanilla (chrono, 10,427)	−2.1	[−3.7, −0.2]	−1.0	1.000
RegMix vs Vanilla (chrono, 1,043)	−6.1	[−8.8, −3.3]	−1.7	0.623
RegMix vs Vanilla (chrono, 2,607)	−11.9	[−18.1, −7.9]	−1.8	0.520
RegMix vs Vanilla (chrono, 5,214)	−14.0	[−21.7, −8.0]	−1.6	0.751
RegMix vs Vanilla (chrono, 10,427)	−6.5	[−10.3, −2.7]	−1.3	1.000

F. Discussion

On this dataset augmentation brought little benefit (RQ1). Its only visible advantage was for very small random training sets: RegMix had the lowest mean RMSE at 30 and 75 examples, but the advantage was not significant after correction, vanished by 150 examples and reversed at 300, where Vanilla training was best. Under chronological evaluation no augmentation method beat Vanilla at any size, and RegMix, now run at all four sizes, was worse than Vanilla at each of them, so we found no evidence that augmentation helps forecasts of a future period. On the second question (RQ2), RegMix was ahead of unrestricted Mixup at 30, 75 and 150 examples (144.4 versus 151.0, 135.5 versus 140.7 and 129.9 versus 131.7) but behind it at 300 (115.7 versus 113.2; difference not significant). The simple global kNN restriction matched RegMix at the two smaller chronological sizes, was better at the two larger ones (by 2.1 and 4.3 bikes per hour), matched it at 150 random examples, was clearly better at 300,

and was behind only at 30 and 75 random examples, so in our runs the learned per-example policy did not clearly beat a single shared neighbourhood size. Validation also tended to prefer weak mixing: the selected Mixup concentration was $\alpha \leq 1$ in all 20 chronological runs, and global kNN selected $k = 1$ in all five runs with 300 examples.

The learned policy stayed close to uniform. Across the 40 RegMix runs, no neighbour option was assigned to more than 40% of the examples; on average the most common option covered 21.2% of examples and the least common 13.6%, compared with 16.7% for an even split over six options. With 40 iterations and 1,500-step candidate models, the controller therefore did not become sharply selective, and part of RegMix's small-sample gain may come from mixing each example with a randomly sized neighbourhood rather than from a learned choice. Separating the two would need a random-policy control, which we did not run, so this remains a hypothesis. On cost (RQ5), the search took about 154 seconds per run (about 3.9 seconds per iteration), almost the same for every setting. At 300 examples RegMix needed 163.8 seconds in total against 20.7 for Vanilla, about 8 times as much, and averaged over all sizes of the reproduction-style protocol about 14 times as much (172 versus 12 seconds). Under the chronological protocol it needed 172.7 seconds against 12.0 for Vanilla at 10,427 rows, about 14 times as much. Training times of the other methods depend on when early stopping triggers and on training Mixup and kNN grids in parallel, so only the large gap to RegMix is a clear conclusion.

The measured results cannot be compared numerically with Table II, because the RMSE values in [15] exceed the target standard deviation of our file (Section III-B). Three further differences matter. First, the original study uses a small random subset, whereas our chronological protocol tests on a later period, so any shift in demand between the training and test periods affects the two protocols differently. Second, our RegMix runs with a much smaller search budget than the one reported in [15] (40 policy-search iterations, whereas the reward curves in [15] run for up to 300 episodes, and fixed-step candidate training), so a smaller improvement than theirs would not by itself contradict their findings. Third, the exact test set and input encoding in [15] are not fully specified, and our input representation differs from a raw 12-feature encoding. For these reasons, our results are not presented as a reproduction of the published numbers.

Limitations. The data come from a single bike-sharing system over two years, so the findings are not generalised to other cities. We used five seeds, which limits statistical power: the Wilcoxon test cannot go below $p = 0.0625$ and Holm correction over 40 comparisons is conservative. RegMix used a reduced budget and produced a near-uniform policy, so our conclusions about RegMix apply to this reduced version. Interpolated examples may be unrealistic even between near neighbours, especially across hours and weather categories that are represented with one-hot or cyclical features. The results depend on the chosen network and on hyperparameters tuned on validation data, and the learning rate was selected once per protocol from a single seed. Timing was measured on one GPU and is affected by early stopping.

IV. Conclusion

In this comparison on the hourly UCI Bike Sharing data, augmentation did not reliably improve demand prediction. Selective nearest-neighbour mixing (our reduced-budget RegMix) gave lower mean error than Vanilla and Mixup only for very small random training sets of 30 and 75 examples, and the advantage at 30 examples (about 9 bikes per hour, winning in all five seeds) did not remain significant after multiple-comparison correction. With 300 random examples Vanilla training was best and RegMix was significantly worse, and under chronological evaluation Vanilla training was best at every size, with RegMix worse than Vanilla at each of the four sizes. RegMix cost about 8 to 14 times the training time of Vanilla. The practical implication is that a plain multilayer perceptron is a strong baseline for hourly demand data once a few hundred records are available, and that augmentation is worth trying mainly with very small labelled histories.

These conclusions apply to one system and to our reduced search budget, under which the learned policy stayed close to uniform. Future work includes running a much longer search (the reward curves in [15] run to about 300 episodes), adding a random-policy control to separate learned from random neighbourhood selection, testing further time periods and other bike-sharing systems, comparing with label-similarity mixing such as C-Mixup [19], using forecasting architectures that model temporal dependence directly, and replacing the repeated model training in policy search with more efficient augmentation-policy methods, a direction also proposed in [15].

Acknowledgment

The authors thank the Department of Computer Engineering, PCCOE, for academic support, and the organisers of the INNOVATE Young Researchers' Conference. The Bike Sharing dataset was provided by H. Fanaee-T through the UCI Machine Learning Repository under a CC BY 4.0 licence.

References

[1] V. Albuquerque, M. S. Dias, and F. Bação, "Machine learning approaches to bike-sharing systems: A systematic literature review," *ISPRS Int. J. Geo-Inf.*, vol. 10, no. 2, 2021, doi: 10.3390/ijgi10020062.

[2] E. Eren and V. E. Uz, "A review on bike-sharing: The factors affecting bike-sharing demand," *Sustain. Cities Soc.*, vol. 54, Art. no. 101882, 2020, doi: 10.1016/j.scs.2019.101882.

[3] A. Kaltenbrunner, R. Meza, J. Grivolla, J. Codina, and R. Banchs, "Urban cycles and mobility patterns: Exploring and predicting trends in a bicycle-based public transport system," *Pervasive Mob. Comput.*, vol. 6, no. 4, pp. 455–466, 2010, doi: 10.1016/j.pmcj.2010.07.002.

[4] C. Shorten and T. M. Khoshgoftaar, "A survey on image data augmentation for deep learning," *J. Big Data*, vol. 6, Art. no. 60, 2019, doi: 10.1186/s40537-019-0197-0.

[5] Q. Wen et al., "Time series data augmentation for deep learning: A survey," in *Proc. 30th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2021, pp. 4653–4660.

[6] B. K. Iwana and S. Uchida, "An empirical survey of data augmentation for time series classification with neural networks," *PLoS ONE*, vol. 16, no. 7, Art. no. e0254841, 2021, doi: 10.1371/journal.pone.0254841.

[7] C. M. Bishop, "Training with noise is equivalent to Tikhonov regularization," *Neural Comput.*, vol. 7, no. 1, pp. 108–116, 1995.

[8] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond empirical risk minimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018.

[9] L. Carratino, M. Cissé, R. Jenatton, and J.-P. Vert, "On mixup regularization," arXiv:2006.06049, 2020.

[10] L. Zhang, Z. Deng, K. Kawaguchi, A. Ghorbani, and J. Zou, "How does mixup help with robustness and generalization?," in *Proc. ICLR*, 2021.

[11] V. Verma et al., "Manifold mixup: Better representations by interpolating hidden states," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2019, pp. 6438–6447.

[12] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, "CutMix: Regularization strategy to train strong classifiers with localizable features," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 6023–6032.

[13] O. Chapelle, J. Weston, L. Bottou, and V. Vapnik, "Vicinal risk minimization," in *Adv. Neural Inf. Process. Syst. (NIPS)*, 2000, pp. 416–422.

[14] H. Guo, Y. Mao, and R. Zhang, "MixUp as locally linear out-of-manifold regularization," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 3714–3722.

[15] S.-H. Hwang and S. E. Whang, "RegMix: Data mixing augmentation for regression," arXiv:2106.03374v4, 2022.

[16] E. D. Cubuk, B. Zoph, D. Mané, V. Vasudevan, and Q. V. Le, "AutoAugment: Learning augmentation strategies from data," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 113–123.

[17] S. Lim, I. Kim, T. Kim, C. Kim, and S. Kim, "Fast AutoAugment," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2019, pp. 6662–6672.

[18] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," in *Proc. ICLR*, 2017.

[19] H. Yao, Y. Wang, L. Zhang, J. Zou, and C. Finn, "C-Mixup: Improving generalization in regression," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2022, arXiv:2210.05775.

[20] H. Fanaee-T, "Bike Sharing [Dataset]," UCI Machine Learning Repository, 2013, doi: 10.24432/C5W894. [Online]. Available: https://archive.ics.uci.edu/dataset/275/bike+sharing+dataset

[21] H. Fanaee-T and J. Gama, "Event labeling combining ensemble detectors and background knowledge," *Prog. Artif. Intell.*, vol. 2, no. 2–3, pp. 113–127, 2014, doi: 10.1007/s13748-013-0040-3.

[22] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Inf. Sci.*, vol. 191, pp. 192–213, 2012.

[23] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015, arXiv:1412.6980.

[24] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 785–794.

[25] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 3146–3154.

[26] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[27] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[28] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[29] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bull.*, vol. 1, no. 6, pp. 80–83, 1945.

[30] B. Efron, "Bootstrap methods: Another look at the jackknife," *Ann. Statist.*, vol. 7, no. 1, pp. 1–26, 1979.

[31] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, 2006.

[32] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.

[33] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2019, pp. 8024–8035.

[34] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[35] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nat. Methods*, vol. 17, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.